

Practical: Air-Quality Data Cleaning Using R

Topic

Loops, Functions, Error Handling, and Advanced Missing Data Handling

Subtopic

Management of NA, NULL, and NaN Values

Estimated Duration

2 Hours

Dataset

Beijing Multi-Site Air Quality Dataset

Dataset source: UCI Machine Learning Repository

[Download the Beijing Multi-Site Air Quality Dataset](#)

For this practical, students should use **only one station CSV file** from the downloaded dataset, such as:

PRSA_Data_Aotizhongxin_20130301-20170228.csv

The selected dataset contains hourly air-quality and weather observations, including:

- PM2.5
- PM10
- SO2
- NO2
- CO
- O3
- TEMP
- PRES
- DEWP
- RAIN
- wd
- WSPM

Some observations contain missing values represented by NA.

Real-World Problem Statement

An environmental monitoring agency collects hourly air-quality and weather readings from different monitoring stations. Due to sensor failure, temporary network interruption, equipment

maintenance, and data-entry problems, some pollutant and weather readings are unavailable or invalid.

Before the dataset can be used for pollution analysis, the agency needs an R program that can:

- Identify missing and undefined values.
- Differentiate between NA, NULL, and NaN.
- Summarize missing values in selected variables.
- Replace missing numerical values using suitable methods.
- Replace missing categorical values.
- Handle errors without terminating the complete program.
- Export a cleaned dataset for further analysis.

Develop an R program to clean the air-quality dataset using loops, user-defined functions, and error-handling mechanisms.

Tasks to Be Performed

Task 1: Import and Inspect the Dataset

Import one station CSV file into R.

Perform the following operations:

1. Display the first six records.
2. Display the structure of the dataset.
3. Display the number of rows and columns.
4. Check whether the dataset contains missing values.
5. Display the total number of missing values.

Use tryCatch() to display a meaningful error message when:

- The file is not found.
- The file cannot be opened.
- The file format is incorrect.

Task 2: Understand NA, NULL, and NaN

Demonstrate the difference between the following values in R:

NA

Represents an unavailable or missing observation.

Example:

```
temperature <- c(28, 30, NA, 32)
```

NULL

Represents an absent or empty R object.

Example:

```
missing_object <- NULL
```

NaN

Represents an undefined numerical result.

Example:

```
undefined_value <- 0 / 0
```

Use the following functions to identify these values:

```
is.na()
```

```
is.null()
```

```
is.nan()
```

Create at least one example of each and display the result.

Task 3: Create a Missing-Value Summary Function

Create a user-defined function named:

```
missing_summary()
```

The function should accept a data frame as input and generate a summary containing:

Variable	Total Records	Missing Values	Missing Percentage
----------	---------------	----------------	--------------------

Apply the function to the following selected variables:

- PM2.5
- PM10
- SO2
- NO2
- TEMP
- WSPM
- wd

The function should generate a warning when a variable contains more than 20% missing values.

Task 4: Identify Invalid Numerical Results

Create a new variable named `pollution_ratio` using:

```
pollution_ratio <- PM2.5 / PM10
```

Check the newly created variable for:

- NA
- NaN
- Positive infinity
- Negative infinity

Use the following functions:

`is.na()`

`is.nan()`

`is.infinite()`

Replace all NaN and infinite values in `pollution_ratio` with NA.

Task 5: Handle Missing Numerical Values Using a Loop

Create a vector containing the names of selected numerical variables:

```
numeric_variables <- c("PM2.5", "PM10", "SO2", "NO2", "TEMP", "WSPM")
```

Use a for loop to perform the following operations for every variable:

1. Check whether the column exists.
2. Count the missing values.
3. Calculate the median using:

```
median(variable, na.rm = TRUE)
```

4. Replace missing values with the median.
5. Display:
 - Variable name
 - Number of missing values before treatment
 - Median used for replacement
 - Number of missing values after treatment

The program should not contain separate repetitive code for every variable.

Task 6: Handle Missing Categorical Values

The `wd` variable represents wind direction.

Create a function named:

```
calculate_mode()
```

The function should calculate the most frequently occurring value in a categorical variable.

Use this function to:

1. Calculate the mode of wd.
2. Replace missing values in wd with the mode.
3. Display the number of missing values before and after replacement.

Task 7: Implement Error Handling

Create a reusable function named:

`clean_variable()`

The function should:

- Accept the dataset and variable name.
- Check whether the variable exists.
- Check whether the variable is numerical.
- Replace missing values using the median.
- Return the cleaned variable.

Use `tryCatch()` to handle the following conditions:

- The variable does not exist.
- A categorical variable is passed instead of a numerical variable.
- The variable contains only missing values.
- The median cannot be calculated.

The program should display an informative message instead of stopping unexpectedly.

Task 8: Compare Missing Values Before and After Cleaning

Prepare a comparison table in the following format:

Variable	Missing Before	Missing After	Values Replaced
----------	----------------	---------------	-----------------

PM2.5			
-------	--	--	--

PM10			
------	--	--	--

SO2			
-----	--	--	--

NO2			
-----	--	--	--

TEMP			
------	--	--	--

WSPM			
------	--	--	--

wd			
----	--	--	--

Interpret whether all selected missing values have been handled successfully.

Task 9: Generate One Visualization

Create a bar chart showing the number of missing values in selected variables:

- Before data cleaning
- After data cleaning

The graph should contain:

- Suitable title
- Variable names on the horizontal axis
- Number of missing values on the vertical axis
- Appropriate legend

Task 10: Export the Cleaned Dataset

Export the cleaned data frame as:

cleaned_air_quality_data.csv

Use:

`write.csv()`

Ensure that row numbers are not included in the exported file.

Expected Deliverables

Students must submit:

1. Complete R script.
2. Missing-value summary table.
3. Comparison table showing missing values before and after cleaning.
4. One missing-value visualization.
5. Cleaned dataset in CSV format.
6. Brief interpretation of approximately 100–150 words.

Expected Learning Outcome

After completing this practical, students will be able to:

- Differentiate between NA, NULL, and NaN in R.
- Identify and summarize missing values.
- Develop reusable functions for data cleaning.
- Apply loops to process multiple variables.
- Handle numerical and categorical missing values.

- Implement error handling using tryCatch().
 - Generate and export a cleaned real-world dataset.
- ALSO UPLOAD THE SUBMISSION ON GIT ACCOUNT AND SHARE LINK OF GIT UPLOAD IN SUBMISSION

I hereby declare that:

1. The assignment / examination has been designed in alignment with the approved curriculum and intended Learning / Course Outcomes.
2. The assessment incorporates the following key elements:
 - o Problem-solving ability
 - o Critical reasoning
3. An AI tool has been used only for reviewing and enhancing assessment quality (where applicable) and not as a replacement for academic judgment.
4. The assignment has been reviewed, finalized, and approved by me, and I take full academic responsibility for its content, rigor, fairness, and integrity.
5. I understand that submission of this declaration is mandatory for all assignments and that it must be presented during the Meeting with the Chief Quality Officer (CQO) or any quality audit.

AI Feedback Scores

Rating: 1 – Lowest | 4 – Highest

Parameter	Score
Real-life Application	4 / 4
Problem-solving Ability	4 / 4
Critical Reasoning	4 / 4
Creativity	4 / 4
Alignment with Learning / Course Outcomes	4 / 4

Estimated Time Required

1 Hour 45 Minutes – 2 Hours

Lab 3: 50–55 Minutes

- Develop reusable functions using if-else
- Implement tryCatch() for exception handling
- Clean invalid values, outliers, and missing entries
- Compare loop-based and vectorized approaches for data processing

- Generate benchmark results and document the developed functions

Lab 4: 55–65 Minutes

- Identify different types of missing data using built-in and external R packages
- Apply suitable imputation and data cleaning strategies
- Validate the cleaned dataset using **skimr**
- Generate missing data reports and visualize missingness patterns

COs Linked with the Assignment

CO1, CO2, CO3

LOs Linked with the Assignment

LO 2.1, LO 2.2, LO 2.3, LO 2.4, LO 3.1, LO 3.2

Justification

This laboratory assignment provides students with practical exposure to one of the most critical stages of the data science lifecycle—**data cleaning, preprocessing, and robust data handling**. The laboratory exercises are based on realistic healthcare and employee survey datasets that simulate challenges commonly encountered by data analysts and data scientists in real-world applications. Students are required to identify invalid observations, detect outliers, manage multiple forms of missing data, implement structured error handling, and validate the quality of cleaned datasets using widely adopted R packages.

The third laboratory develops programming proficiency by requiring students to design reusable functions using conditional statements, implement exception handling through **tryCatch()**, and compare conventional iterative solutions with efficient vectorized implementations. These activities strengthen algorithmic thinking, debugging skills, code optimization, and computational efficiency while encouraging students to evaluate different implementation strategies.

The fourth laboratory extends these concepts by introducing advanced missing data management techniques. Students distinguish between **NA**, **NULL**, and **NaN**, apply appropriate imputation methods, validate processed datasets using **skimr**, and visualize missingness patterns to assess data quality. Such activities closely reflect industry-standard data preprocessing workflows that precede statistical analysis and machine learning model development.

The assignment demonstrates strong alignment with **CO1** by reinforcing fundamental R programming concepts, functions, control structures, and data handling techniques. It strongly supports **CO2** through practical implementation of data manipulation, preprocessing, and analytical reasoning using modern R packages. Furthermore, it substantially contributes to **CO3** by requiring students to import, clean, transform, validate, and prepare datasets for subsequent statistical analysis and machine learning tasks. The inclusion of benchmarking, reusable function development, comprehensive documentation, and reproducible programming practices further enhances students' professional coding skills and critical thinking abilities.

Overall, the assignment effectively bridges theoretical programming concepts with practical data engineering techniques, providing a strong foundation for advanced statistical computing and machine learning modules.

AI Tool Used

ChatGPT

Prompt(s) Used

"Evaluate the given R Programming laboratory assignment using a 1–4 rubric for Real-life Application, Problem-solving Ability, Critical Reasoning, Creativity, and Alignment with Learning / Course Outcomes. Identify the estimated time required, map the relevant Course Outcomes (CO1–CO6) and Learning Outcomes (LOs), and justify the ratings based on the R Programming syllabus covering control flow, functions, exception handling, vectorized programming, missing data handling, data preprocessing, validation, and data quality assessment."